

C++ Notes

Error Handling

Adam HAMOUC

May 2026

1. Les trois approches

- **Codes de retour** : style C. Facile à ignorer, peu de contexte. Préserve le déroulement du programme.
- **Exceptions** : riche contexte, impossible à ignorer. Coûteux si throw (stack unwinding). Idéal pour les erreurs rares/critiques.
- **assert** : pour les bugs de programmation uniquement. Désactive en release (NDEBUG). Stoppe toujours le programme.

```
// Code de retour : facile à oublier de vérifier
int load(const char* path, Texture** out);

// Exception : contexte riche, remonte automatiquement la pile
Texture* load(const char* path) {
    if (!fileExists(path))
        throw AssetNotFoundException(path); // contexte clair
    return createTexture(path);
}

// assert : invariant qui ne doit JAMAIS être violé
void draw(Mesh* mesh) { assert(mesh != nullptr); }
```

2. Exceptions — mécanisme

Une exception remonte automatiquement la pile d'appels (**stack unwinding**) jusqu'au premier **catch** compatible. Pendant ce processus, tous les destructeurs des objets locaux sont appelés — RAII garantit donc zéro fuite mémoire même en cas d'exception.

Règle absolue : ne jamais throw dans un destructeur. Si deux exceptions sont en vol simultanément, `std::terminate()` est appelé. Depuis C++11, tous les destructeurs sont implicitement **noexcept**.

```
try {
    auto mesh = std::make_unique<Mesh>(); // alloue
    auto texture = std::make_unique<Texture>(); // alloue
    throw std::runtime_error("oops");
    // mesh et texture détruits automatiquement par RAII -- zéro fuite
} catch (const std::runtime_error& e) {
    log(e.what()); // programme continue
} catch (const std::exception& e) {
    // toutes les autres exceptions standard
} catch (...) {
    // tout attraper -- dernier recours
}

// Re-throw : propage l'exception originale telle quelle
try { loadShaders(); }
catch (const ShaderCompilationError& e) {
    log(e.what()); log(e.getLog());
    throw; // re-throw sans argument : préserve le type et le message
}
```

3. Exceptions custom

```
class EngineException : public std::exception {
public:
```

```

    explicit EngineException(std::string msg) : message(std::move(msg)) {}
    const char* what() const noexcept override { return message.c_str(); }
private:
    std::string message;
};

class ShaderCompilationError : public EngineException {
public:
    ShaderCompilationError(std::string name, std::string log)
        : EngineException("Shader failed: " + name)
        , compilationLog(std::move(log)) {}
    const std::string& getLog() const { return compilationLog; }
private:
    std::string compilationLog;
};

```

4. Alternatives pour le hot path

Les exceptions ont un cout negligeable en cas normal mais **couteux si throw** (stack unwinding). Ne jamais throw dans la boucle de rendu, les systemes ECS per-frame, ou tout hot path a 60 fps.

```

// std::optional : fonction qui peut ne pas retourner de valeur
std::optional<Texture*> findTexture(const std::string& name) {
    auto it = cache.find(name);
    if (it == cache.end()) return std::nullopt;
    return it->second;
}

if (auto tex = findTexture("rock.png")) (*tex)->bind();

// std::expected (C++23) : transporte le resultat OU l'erreur
std::expected<Texture*, std::string> loadTexture(const std::string& path) {
    if (!fileExists(path)) return std::unexpected("Not found: " + path);
    return createTexture(path);
}

auto result = loadTexture("rock.png");
if (result) result.value()->bind();
else log(result.error());

```

5. Regle pratique engine dev

Initialisation (load, compile, init)	throw — erreur rare, contexte critique
Hot path (update, render, physics)	optional / expected — pas de throw
Bug de programmation (invariant)	assert — desactive en release

Si throw dans l'init et qu'on ne peut pas se recuperer → **re-throw** apres log.

```

// Init : exception OK
void Engine::init() {
    try { loadAllShaders(); }
    catch (const ShaderCompilationError& e) {
        logger.critical(e.what()); logger.critical(e.getLog());
        throw; // moteur ne peut pas demarrer sans ses shaders
    }
}

// Hot path : jamais d'exception
void Engine::update(float dt) {
    for (auto& entity : entities) {
        if (auto t = getComponent<Transform>(entity))
            t->position += t->velocity * dt; // optional, pas de throw
    }
}

```